Module 3 — Dynamics & Control: Formula Summary

Lessons 11–15 · Quick reference · Angles in radians for computations

Lesson 11 — Fundamental mechanics

```
Linear dynamics:       F_net = m*a
Rotation, fixed axis:  tau_net = I*alpha
Horizontal arm payload gravity torque magnitude: tau_g = m*g*L
Angle theta from horizontal: gravity torque magnitude = |m*g*L*cos(theta)|
```

Signs matter: in a CCW-positive model, downward gravity contributes -m*g*L*cos(theta) to net torque. A holding motor must counter it. These expressions model a point payload and a massless arm unless otherwise stated.

Units: F in N; m in kg; I in kg·m²; alpha in rad/s²; torque in N·m.

Lesson 12 — Single-joint equation of motion

```
I*theta_ddot + b*theta_dot + m*g*L*cos(theta) = tau_motor
theta_ddot = (tau_motor - b*theta_dot - m*g*L*cos(theta))/I
omega_next = omega + alpha*dt
theta_next = theta + omega_next*dt   # semi-implicit Euler
```

Example with m=1 kg, L=0.5 m, g=9.81 m/s²: horizontal holding torque = 4.905 N·m. Constant gravity compensation works only for the specified equilibrium angle.

Lesson 13 — Feedback control

```
Position error: e = theta_desired - theta
P:   tau = Kp*e
PD (stationary target): tau = Kp*e - Kd*theta_dot
PID: tau = Kp*e + Ki*integral(e dt) + Kd*de/dt
PD plus gravity compensation:
    tau = Kp*e - Kd*theta_dot + m*g*L*cos(theta)
```

For a changing target, derivative-on-error and derivative-on-measurement are not identical. The code examples assume a constant target.

Lesson 14 — Tuning and stability

After ideal gravity cancellation for a constant target:

```
I*theta_ddot + (b+Kd)*theta_dot + Kp*(theta-theta_desired) = 0
Natural frequency: omega_n = sqrt(Kp/I)
Damping ratio: zeta = (b+Kd)/(2*sqrt(I*Kp))
```

0 < zeta < 1: underdamped; zeta = 1: critically damped; zeta > 1: overdamped. For the ideal continuous-time model, Kp > 0 and b+Kd > 0 imply asymptotic stability. Real actuators, time delays, saturation, and discrete integration can change outcomes.

Example (I=0.5, b=0.2, Kp=12): Kd=0.3 gives zeta≈0.102; Kd=4.7 gives zeta≈1.000; Kd=10 gives zeta≈2.082.

Lesson 15 — Saturation and anti-windup

```
tau_actual = clip(tau_requested, -tau_max, +tau_max)
Integral state: E = integral(e dt)
Basic integral clamp: E = clip(E, -E_max, +E_max)
```

Integral windup is associated with an integrating controller continuing to accumulate error while the actuator cannot deliver requested torque. Our Lesson 15 PD example contains no integral term: it demonstrates torque saturation, not windup directly.

Connection across lessons

Kinematics chooses desired joint motion; dynamics predicts the response to applied torque; feedback adjusts torque based on state error; tuning shapes the response; actuator limits constrain feasible motion. These foundations support articulated Isaac Sim robots and later learned control policies.